

SKILL WEEK - 4

SESSION 7 :

Task -1

Process Synchronization and Child Process Monitoring Using waitpid()

CODE

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("Child process started. PID = %d\n", getpid());

        sleep(3);

        printf("Child process completed.\n");

        return 10;
    }
    else
    {
        // Parent process
        printf("Parent is waiting for child process...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child terminated normally.\n");
            printf("Child exit status = %d\n", WEXITSTATUS(status));
        }

        printf("Parent process completed.\n");
    }

    return 0;
}
```

OUTPUT

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week4
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week4$ nano task1.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week4$ gcc task1.c -o task1
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week4$ ./task1
Parent process started. PID = 540
Parent is waiting for child process...
Child process started. PID = 541
Child process completed.
Child terminated normally.
Child exit status = 10
Parent process completed.
```

OBSERVATION

The program creates a child process using `fork()`. The parent process uses `waitpid()` to wait for the child process to finish before continuing its own execution. The child process performs its task and terminates with an exit status. The parent then checks whether the child has terminated normally and displays its exit status using `WIFEXITED()` and `WEXITSTATUS()`. This shows how `waitpid()` helps the parent monitor the child process and maintain proper synchronization between the two processes.

Task 2

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

CODE

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }

    char *path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH = %s\n", path);

    char *path_copy = strdup(path);
    char *dir = strtok(path_copy, ":");
    char full_path[1024];
    int found = 0;

    while (dir != NULL)
    {
        snprintf(full_path, sizeof(full_path), "%s/%s", dir, argv[1]);

        if (access(full_path, F_OK) == 0)
        {
            printf("\nFile found: %s\n", full_path);

            if (access(full_path, X_OK) == 0)
                printf("Execute permission: YES\n");
            else
                printf("Execute permission: NO\n");

            printf("Command resolved successfully.\n");
            found = 1;
            break;
        }

        dir = strtok(NULL, ":");
    }

    if (!found)
    {
        printf("\nCommand '%s' not found in PATH.\n", argv[1]);
    }

    free(path_copy);

    return 0;
}
```

OUTPUT

```
Parent process completed.
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ nano task1.c
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ nano task2.c
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ gcc task2.c -o task2
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ ./task2
Usage: ./task2 <command>
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ ./task2 pwd
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinux.2.7.10.0_x64_8wekyb3d8bbwe:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/windows/system32:/mnt/c/windows:/mnt/c/windows/System32/Wbem:/mnt/c/windows/System32/WindowsPowerShell/v1.0:/mnt/c/windows/System32/OpenSSH:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/Program Files/Git/cmd:/mnt/c/ProgramData/chocolatey/bin:/mnt/c/Program Files/nodejs:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/anany/AppData/Local/Programs/Python/Python311/Scripts:/mnt/c/Users/anany/AppData/Local/Programs/Python/Python311:/mnt/c/Users/anany/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/anany/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/anany/AppData/Roaming/npm:/snap/bin

File found: /usr/bin/pwd
Execute permission: YES
Command resolved successfully.
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ |
```

```
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ ./task2 ananya
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinux.2.7.10.0_x64_8wekyb3d8bbwe:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/windows/system32:/mnt/c/windows:/mnt/c/windows/System32/Wbem:/mnt/c/windows/System32/WindowsPowerShell/v1.0:/mnt/c/windows/System32/OpenSSH:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/Program Files/Git/cmd:/mnt/c/ProgramData/chocolatey/bin:/mnt/c/Program Files/nodejs:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/anany/AppData/Local/Programs/Python/Python311/Scripts:/mnt/c/Users/anany/AppData/Local/Programs/Python/Python311:/mnt/c/Users/anany/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/anany/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/anany/AppData/Roaming/npm:/snap/bin

Command 'ananya' not found in PATH.
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ |
```

OBSERVATION

The program retrieves the PATH environment variable and searches through its directories to find the given command. It checks whether the file exists and whether it has execute permission using `access()`. If the command is found, its path and permission status are displayed. If the command is not found, an appropriate message is displayed. Thus, the program successfully demonstrates how commands can be located and verified using the PATH variable.

SESSION 8 :

Task -1

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

CODE

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int main()
{
    char input[200], output[500];
    int i = 0, j = 0;

    printf("Enter a string: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    while (input[i] != '\0')
    {
        if (input[i] == '$' &&
            (isalpha(input[i + 1]) || input[i + 1] == '_'))
        {
            char var[100];
            int k = 0;

            i++;

            while (isalnum(input[i]) || input[i] == '_')
                var[k++] = input[i++];

            var[k] = '\0';

            char *value = getenv(var);

            if (value != NULL)
            {
                strcpy(&output[j], value);
                j += strlen(value);
            }
            else
            {
                printf("Warning: Undefined variable: %s\n", var);
            }
        }
        else
        {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';

    printf("Original : %s\n", input);
    printf("Expanded : %s\n", output);

    return 0;
}
```

OUTPUT

```
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ nano task3.c
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ gcc task3.c -o task3
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ ./task3
Enter a string: Hello
Original : Hello
Expanded : Hello
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ ./task3
Enter a string: Hello $USER
Original : Hello $USER
Expanded : Hello ananya
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ ./task3
Enter a string: Hello $abc
Warning: Undefined variable: abc
Original : Hello $abc
Expanded : Hello
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week4$ |
```

OBSERVATION

The program successfully detects environment variable references written using the \$ symbol and extracts the variable name from the input string. It uses `getenv()` to retrieve the value of the environment variable and replaces the variable reference with its actual value. If the specified variable is not defined, the program displays a warning message instead of replacing it. The program therefore demonstrates how environment variables can be detected, retrieved, and expanded in an input string, similar to the way a shell handles environment variables,

Task -2

To Identify Built-ins, Create Dispatch Table, Execute In-Process Built-ins, Handle Invalid Commands, Maintain State, Test Dispatch Logic

CODE

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main()
{
    char input[200];
    char *cmd, *arg;
    char cwd[200];

    while (1)
    {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        cmd = strtok(input, " ");
        arg = strtok(NULL, " ");

        if (cmd == NULL)
            continue;

        if (strcmp(cmd, "pwd") == 0)
        {
            getcwd(cwd, sizeof(cwd));
            printf("%s\n", cwd);
        }
        else if (strcmp(cmd, "cd") == 0)
        {
            if (arg == NULL)
                printf("cd: missing argument\n");
            else if (chdir(arg) != 0)
                perror("cd");
        }
        else if (strcmp(cmd, "echo") == 0)
        {
            while (arg != NULL)
            {
                printf("%s ", arg);
                arg = strtok(NULL, " ");
            }
            printf("\n");
        }
        else if (strcmp(cmd, "exit") == 0)
        {
            printf("Exiting shell...\n");
            exit(0);
        }
        else
        {
            printf("Command not found: %s\n", cmd);
        }
    }

    return 0;
}
```

OUTPUT

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week4
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week4$ nano task4.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week4$ gcc task4.c -o task4
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week4$ ./task4
myshell> pwd
/mnt/d/OSSP_Programs/Skill/Week4
myshell> echo Hello World
Hello World
myshell> cd/tmp
Command not found: cd/tmp
myshell> pwd
/mnt/d/OSSP_Programs/Skill/Week4
myshell> abc
Command not found: abc
myshell> exit
Exiting shell...
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week4$ |
```

OBSERVATION

The program successfully identifies and executes basic built-in commands such as `pwd`, `cd`, `echo`, and `exit`. It uses `strcmp()` to identify the entered command and performs the corresponding operation. The `cd` command changes the current working directory, while `pwd` displays the updated directory. The program also handles invalid commands by displaying a “Command not found” message and terminates the shell when `exit` is entered. This demonstrates how built-in commands can be handled within a shell program.